Stage 3: Context-Aware Fine-Tuning with Rotary Position Embeddings

Technical Report on GraDeT-HTR Context-Aware Extension

GraDeT-HTR Project

March 2026

Abstract

This report details the Stage 3 fine-tuning of the GraDeT-HTR Bengali handwritten text recognition system with context-aware training and Rotary Position Embeddings (RoPE). Building on a Stage 2 word-level pretrained model, we replace absolute positional embeddings with RoPE, introduce a dual-path training objective that conditions recognition on the previously recognized word, and employ a margin penalty to ensure context consistently benefits recognition. After 7 epochs of fine-tuning on an NVIDIA L4 GPU, the model achieves 92.02% token-level accuracy with context versus 91.95% without, with context loss consistently lower than isolation loss across all epochs, demonstrating that inter-word context improves recognition quality.

Contents

1 Introduction **2**

2 Background **2**
2.1 Base Architecture: DTrOCR . . . 2
2.2 Rotary Position Embeddings (RoPE) . . . 2

3 Architecture Modifications **3**
3.1 Context-Aware Sequence Design . . . 3
3.2 RoPE Integration . . . 3
3.3 RotaryEmbedding Implementation . . . 4

4 Weight Conversion: GPT-2 to RoPE **4**
4.1 Conv1D to Linear Transposition . . . 4
4.2 Splitting Combined Attention Projections . . . 5
4.3 MLP Index Mapping . . . 5
4.4 Dropped and Skipped Parameters . . . 5

5 Dual-Path Training Objective **5**
5.1 Motivation . . . 5
5.2 Loss Formulation . . . 6
5.3 Gradient Analysis . . . 6

6 Data Pipeline **6**
6.1 Dataset Structure . . . 6
6.2 Context Construction . . . 7
6.3 Train/Test Split . . . 7

7 Training Configuration **7**
7.1 Hyperparameters . . . 7
7.2 Training Procedure . . . 7

8 Results **7**
8.1 Training Progression . . . 7
8.2 Training Curves Analysis . . . 7
8.3 Context Benefit Analysis . . . 8

9 Implementation Details **9**
9.1 Processor Fix: Fixed-Length Context Padding . . . 9
9.2 RoPE Sequence Length Adjustment . . . 10
9.3 Memory Optimization . . . 10

10 Discussion **11**
10.1 Effectiveness of Context-Aware Training . . . 11
10.2 Role of the Margin Penalty . . . 11
10.3 RoPE vs. Absolute Positional Embeddings . . . 11
10.4 Limitations and Future Work . . . 11

11 Conclusion **12**

1 Introduction

Handwritten text recognition (HTR) systems typically process words in isolation—each word image is independently fed to a recognition model without awareness of surrounding text. However, in real handwritten documents, words appear in sentences where linguistic context provides strong disambiguation cues. A Bengali word that appears ambiguous in isolation may become unambiguous when the preceding word is known.

GraDeT-HTR is a Bengali HTR system combining YOLOv5-based line and word detection (BN-DRISHTI) with a custom decoder-only transformer (DTrOCR) using grapheme-based tokenization. The system processes documents through a three-stage pipeline: line segmentation, word segmentation, and text extraction.

This report documents **Stage 3**: the fine-tuning phase where we:

1. Replace absolute positional embeddings with **Rotary Position Embeddings** (RoPE) for improved relative position encoding.
2. Introduce a **dual-path training objective** that simultaneously trains the model with and without context from the previous word.
3. Apply a **margin penalty** to ensure the context-aware path outperforms the isolation path.
4. Convert Stage 2 pretrained weights from GPT-2 format to the new RoPE architecture.

2 Background

2.1 Base Architecture: DTrOCR

The DTrOCR model is a vision-language transformer that combines:

- **ViT Patch Embeddings**: Converts 32×128 RGB images into 128 patch tokens via 4×8 convolutional patches.
- **Token Embedding**: Maps Bengali grapheme tokens (vocabulary size 1420) to 768-dimensional vectors.
- **12 GPT-2 Transformer Blocks**: Each with multi-head self-attention (12 heads, 64-dim per head) and a feed-forward network (3072 inner dimension).
- **Language Model Head**: Linear projection from hidden states to vocabulary logits for autoregressive generation.

The input sequence is constructed as:

$$[\text{IMAGE_PATCHES}_{1..128}, \text{BOS}, \text{target_tokens}_{1..n}]$$

The model is trained with causal attention and cross-entropy loss, where predictions start after the image patches and BOS token.

2.2 Rotary Position Embeddings (RoPE)

Rotary Position Embeddings, introduced by Su et al. in the RoFormer paper [1], encode positional information by rotating query and key vectors in attention. Unlike absolute positional embeddings (which are added to token embeddings), RoPE is applied multiplicatively inside the attention mechanism.

The key equations from the paper are:

Frequency computation (Eq. 15):

$$\theta_i = 10000^{-2(i-1)/d}, \quad i \in \{1, 2, \ldots, d/2\} \tag{1}$$

where d is the head dimension (64 in our model).
Efficient rotation (Eq. 34):

$$\mathbf{R}_m \mathbf{x} = \mathbf{x} \odot \cos(m\boldsymbol{\theta}) + \text{rotate_half}(\mathbf{x}) \odot \sin(m\boldsymbol{\theta}) \tag{2}$$

where m is the position index and rotate_half($\mathbf{x}$) splits the vector into two halves and negates the second:

$$\text{rotate_half}([x_1, \ldots, x_{d/2}, x_{d/2+1}, \ldots, x_d]) = [-x_{d/2+1}, \ldots, -x_d, x_1, \ldots, x_{d/2}]$$

Advantages of RoPE over absolute positional embeddings:

- **Relative position awareness**: The attention score between positions m and n depends only on $m - n$, enabling better generalization to varying sequence lengths.
- **No learnable parameters**: Position information is encoded through deterministic rotation matrices, reducing the parameter count.
- **Decaying long-range dependency**: The inner product between RoPE-encoded vectors naturally decays with increasing relative distance, providing an inductive bias for locality.

3 Architecture Modifications

3.1 Context-Aware Sequence Design

In Stage 3, the input sequence is extended with a fixed-length context block representing the previous word's text. The full sequence becomes:

$$\underbrace{[\text{ctx_tokens, SEP, PAD}, \ldots]}_{\text{20 tokens (fixed)}} \underbrace{[\text{IMAGE_PATCHES}_{1..128}]}_{\text{128 tokens}} \underbrace{[\text{BOS, target}_{1..n}]}_{\text{up to 128 tokens}}$$

The total maximum sequence length is $20 + 128 + 128 = 276$ tokens.
Key design decisions:

- **Fixed-length context block** (20 tokens): All samples have identical sequence lengths regardless of whether context is available, enabling efficient batched training. Empty context (first word of a sentence) uses all-padding tokens with zeroed attention masks.
- **SEP token**: The EOS token is reused as a separator between context and image patches, signaling the boundary between linguistic context and visual content.
- **Sentence boundary respect**: Context is only provided from the previous word within the same line. At sentence boundaries (new line_id), context is empty to prevent cross-sentence leakage.

3.2 RoPE Integration

The RoPE-enhanced architecture replaces three components from the original GPT-2-based model:

1. **Positional embedding removed**: The learnable `nn.Embedding(256, 768)` for absolute positions is dropped entirely. Position information is instead encoded through rotation matrices applied inside attention.

2. **GPT2Attention → RoPEAttention**: The combined `c_attn` Conv1D projection (which computes Q, K, V in a single operation) is replaced with separate `q_proj`, `k_proj`, `v_proj` Linear layers. RoPE is applied to Q and K after projection but before the attention computation:

$$\mathbf{q}_m = \mathbf{R}_m \cdot W_Q \mathbf{h}_m \tag{3}$$

$$\mathbf{k}_n = \mathbf{R}_n \cdot W_K \mathbf{h}_n \tag{4}$$

$$\text{attn}(m, n) = \frac{\mathbf{q}_m^\top \mathbf{k}_n}{\sqrt{d}} \tag{5}$$

3. **GPT2Block → RoPEBlock**: The transformer block maintains the same Pre-LN architecture (LayerNorm before attention and MLP, with residual connections), but uses RoPEAttention internally. The MLP uses `nn.Sequential` with GELU activation (`approximate='tanh'`) matching GPT-2's `gelu_new`.

3.3 RotaryEmbedding Implementation

The `RotaryEmbedding` module precomputes cosine and sine tables for all positions up to 276 (the maximum sequence length with context):

Listing 1: RotaryEmbedding initialization

```python
class RotaryEmbedding(nn.Module):
    def __init__(self, dim, max_position_embeddings=276, base=10000.0):
        super().__init__()
        # theta_i = base^(-2(i-1)/d) for i in [1, ..., d/2]
        inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
        self.register_buffer("inv_freq", inv_freq)

        # Precompute cos/sin for all positions
        t = torch.arange(max_position_embeddings).float()
        freqs = torch.outer(t, inv_freq)      # [max_len, dim/2]
        emb = torch.cat((freqs, freqs), dim=-1)  # [max_len, dim]
        self.register_buffer("cos_cached", emb.cos())
        self.register_buffer("sin_cached", emb.sin())
```

The rotation is applied efficiently using element-wise operations (Equation 2), avoiding explicit matrix construction:

Listing 2: Applying RoPE to query and key tensors

```python
def apply_rotary_pos_emb(q, k, cos, sin):
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    return q_embed, k_embed
```

4 Weight Conversion: GPT-2 to RoPE

The Stage 2 model uses GPT-2-style transformer blocks with Conv1D layers. Converting these weights to the RoPE architecture requires several transformations:

4.1 Conv1D to Linear Transposition

GPT-2 uses `Conv1D` layers which store weights as (d_{in}, d_{out}), while PyTorch's `nn.Linear` stores weights as (d_{out}, d_{in}). All weight matrices must be transposed:

GPT-2 Layer	Shape	Transposed Shape
`attn.c_attn.weight`	$(768, 2304)$	$(2304, 768)$
`attn.c_proj.weight`	$(768, 768)$	$(768, 768)$
`mlp.c_fc.weight`	$(768, 3072)$	$(3072, 768)$
`mlp.c_proj.weight`	$(3072, 768)$	$(768, 3072)$

4.2 Splitting Combined Attention Projections

GPT-2's `c_attn` computes Q, K, V in a single $(768, 2304)$ weight matrix. After transposition to $(2304, 768)$, this is split into three $(768, 768)$ matrices:

Listing 3: Splitting combined c_attn into separate Q, K, V projections

```
w = value.t()  # Conv1D (768, 2304) -> (2304, 768)
h = w.shape[0] // 3  # = 768
q_proj.weight = w[:h]       # (768, 768)
k_proj.weight = w[h:2*h]    # (768, 768)
v_proj.weight = w[2*h:]     # (768, 768)
```

4.3 MLP Index Mapping

GPT-2's named MLP layers (`c_fc`, `c_proj`) map to `nn.Sequential` integer indices:

GPT-2 Key	RoPE Key
`mlp.c_fc.weight`	`mlp.0.weight`
`mlp.c_fc.bias`	`mlp.0.bias`
`mlp.c_proj.weight`	`mlp.2.weight`
`mlp.c_proj.bias`	`mlp.2.bias`

(Index 1 is the GELU activation, index 3 is dropout—neither has learnable parameters.)

4.4 Dropped and Skipped Parameters

The following parameters from Stage 2 are intentionally skipped:

- `transformer.positional_embedding.weight`: RoPE replaces absolute positional embeddings. This $(256, 768)$ matrix is discarded.
- `attn.bias` and `attn.masked_bias`: GPT-2's causal mask buffers are unnecessary—SDPA (Scaled Dot-Product Attention) handles causality internally.

The only **missing keys** after conversion are the RoPE buffers (`inv_freq`, `cos_cached`, `sin_cached`), which are computed deterministically during model initialization and require no learned weights.

5 Dual-Path Training Objective

5.1 Motivation

The training objective must achieve two goals simultaneously:

1. The model should **benefit from context** when it is available (previous word helps recognition).
2. The model should **not degrade** when context is absent (first word of a sentence, or inference without context).

A naive approach of training only with context risks the model becoming dependent on it, degrading performance when context is unavailable.

5.2 Loss Formulation

Each training sample is processed through **two forward passes** using the same image but different input sequences:

Context path: The input includes the previous word's tokens.

$$\mathcal{L}_{ctx} = -\frac{1}{T}\sum_{t=1}^{T} \log P(y_t \mid y_{<t}, \mathbf{I}, \mathbf{c})$$

Isolation path: The input uses all-padding context (equivalent to no previous word).

$$\mathcal{L}_{iso} = -\frac{1}{T}\sum_{t=1}^{T} \log P(y_t \mid y_{<t}, \mathbf{I})$$

where $\mathbf{I}$ is the image, $\mathbf{c}$ is the context tokens, and y_t are the target grapheme tokens.

Combined loss with margin penalty:

$$\mathcal{L} = \mathcal{L}_{ctx} + \mathcal{L}_{iso} + \lambda \cdot \max(0, \mathcal{L}_{ctx} - \mathcal{L}_{iso}) \tag{6}$$

The three terms serve distinct purposes:

- $\mathcal{L}_{ctx}$: Drives the model to use context effectively.
- $\mathcal{L}_{iso}$: Maintains performance without context.
- $\lambda \cdot \max(0, \mathcal{L}_{ctx} - \mathcal{L}_{iso})$: The **margin penalty** activates *only when context hurts*—i.e., when the context path has higher loss than the isolation path. This term pushes $\mathcal{L}_{ctx}$ below $\mathcal{L}_{iso}$, ensuring context is always beneficial.

In our experiments, we set $\lambda = 0.7$.

5.3 Gradient Analysis

When $\mathcal{L}_{ctx} > \mathcal{L}_{iso}$ (context is hurting), the margin term is active and the effective gradient becomes:

$$\nabla\mathcal{L} = (1+\lambda)\nabla\mathcal{L}_{ctx} + (1-\lambda)\nabla\mathcal{L}_{iso}$$

This amplifies the gradient from the context path (factor 1.7) while dampening the isolation path gradient (factor 0.3), strongly pushing the model to improve its use of context.

When $\mathcal{L}_{ctx} \leq \mathcal{L}_{iso}$ (context is already helping), the margin is zero and:

$$\nabla\mathcal{L} = \nabla\mathcal{L}_{ctx} + \nabla\mathcal{L}_{iso}$$

Both paths receive equal gradient, maintaining balanced optimization.

6 Data Pipeline

6.1 Dataset Structure

The training data consists of word-level crops of Bengali handwritten text, organized with JSON metadata files containing:

- `output_path`: Path to the cropped word image (32×128 pixels).
- `text`: Ground truth Bengali text transcription.
- `line_id`: Identifier for the source line, used to maintain sentence boundaries.
- `word_index`: Position of the word within its line.

6.2 Context Construction

The `ContextAwareDataset` class builds the previous-word context (`prev_text`) respecting sentence boundaries:

- If the current word and previous word share the same `line_id`, the previous word's text becomes the context.
- At sentence boundaries (different `line_id` or first word), context is an empty string.
- The dataset is split by `line_id` groups to prevent context leakage across train/test boundaries.

Each `__getitem__` call returns **two sets of inputs**: one with context (`prev_text`) and one without (empty string). Both are processed by the `DTrOCRProcessor` which:

1. Tokenizes the context text and truncates to max_context_length $- 1 = 19$ tokens.
2. Appends a SEP token and pads to exactly 20 tokens.
3. For empty context: creates a 20-token all-padding block with zeroed attention mask.
4. Concatenates: $[\text{context_block}_{20}] + [\text{target_tokens}]$.
5. The model's forward pass then reorders to: $[\text{context}_{20}, \text{IMAGE}_{128}, \text{target}]$.

6.3 Train/Test Split

The dataset is split with `test_size=0.05` (95% train, 5% test), stratified by `line_id` groups to keep entire sentences together. After splitting, `prev_text` is rebuilt within each split to prevent cross-split context contamination.

7 Training Configuration

7.1 Hyperparameters

7.2 Training Procedure

The training loop (Algorithm 1) processes each batch with two forward passes (context and isolation), computes the dual-path loss with margin penalty, and updates parameters with AdamW.

8 Results

8.1 Training Progression

Table 2 presents the complete epoch-by-epoch training and validation metrics.

8.2 Training Curves Analysis

Figure 1 shows the training progression across all three metrics.

Key observations from the training curves:

1. **Rapid convergence**: The total training loss drops from 2.82 to 0.25 over 7 epochs, with the steepest descent in epochs 1–3. This indicates effective transfer learning from the Stage 2 pretrained weights.

Table 1: Stage 3 training hyperparameters

Parameter	**Value**
Pretrained weights	Stage 2 (`final_stage2_model.pth`)
Optimizer	AdamW
Learning rate	5×10^{-6}
Weight decay	0.01
Batch size	16
Epochs	7
Margin penalty λ	0.7
Gradient clipping	max norm 1.0
Mixed precision	bfloat16 (autocast)
Model architecture	
Hidden size	768
Attention heads	12
Transformer layers	12
Head dimension	64
MLP inner dimension	3072
Vocabulary size	1420
Image size	32×128
Patch size	4×8
Max context length	20
Max sequence length	276
Total parameters	$\sim$85M
Infrastructure	
GPU	NVIDIA L4 (22 GB)
Time per epoch	$\sim$27 minutes
Total training time	$\sim$3.2 hours

2. **Consistent context advantage**: At every single epoch, the context validation loss is lower than the isolation validation loss. The context path starts with a 1.86% advantage at epoch 1 and maintains a 3.0% advantage at epoch 7.

3. **No overfitting**: Validation losses continue to decrease through epoch 7 (0.2668 context, 0.2751 isolation), closely tracking training losses. The gap between training and validation losses remains small and stable.

4. **Accuracy saturation**: Validation accuracy increases from 81.1% (epoch 1) to 92.0% (epoch 7), with diminishing returns after epoch 5. The model approaches its performance ceiling on this dataset.

8.3 Context Benefit Analysis

Table 3 quantifies the epoch-by-epoch improvement from context.

Final metrics after 7 epochs:

- Context validation loss: **0.2668** vs. Isolation: **0.2751** (3.0% improvement)
- Context validation accuracy: **92.02%** vs. Isolation: **91.95%** (0.07pp improvement)
- Context benefit is consistent: negative loss difference at *every* epoch, confirming the margin penalty ($\lambda = 0.7$) successfully enforces $\mathcal{L}_{ctx} < \mathcal{L}_{iso}$.

Algorithm 1 Stage 3 Dual-Path Training Loop

Require: Pretrained Stage 2 weights, training data with context annotations
Ensure: Fine-tuned RoPE model with context-aware recognition

Convert Stage 2 weights from GPT-2 format to RoPE format
Load converted weights into DTrOCRLMHeadModelRoPE
for epoch = 1 to 7 **do**
  for each batch $(\mathbf{I}, \mathbf{y}, \mathbf{c})$ in training set **do**
    $\mathcal{L}_{ctx} \leftarrow \text{forward}(\mathbf{I}, \mathbf{y}, \text{context} = \mathbf{c})$ ▷ Context path
    $\mathcal{L}_{iso} \leftarrow \text{forward}(\mathbf{I}, \mathbf{y}, \text{context} = \varnothing)$ ▷ Isolation path
    $\text{margin} \leftarrow \max(0, \mathcal{L}_{ctx} - \mathcal{L}_{iso})$
    $\mathcal{L} \leftarrow \mathcal{L}_{ctx} + \mathcal{L}_{iso} + 0.7 \cdot \text{margin}$
    Backpropagate $\mathcal{L}$, clip gradients, update with AdamW
  end for
  Validate on test set (both paths)
  Save checkpoint to local storage and Google Drive
end for

Table 2: Epoch-by-epoch training and validation metrics for Stage 3

Epoch	Training Loss			Validation Loss		Validation Accuracy	
	Total	Context	Isolation	Context	Isolation	Context	Isolation
1	2.8196	1.4013	1.4066	0.6507	0.6693	0.8112	0.8067
2	0.8818	0.4310	0.4459	0.4342	0.4504	0.8731	0.8682
3	0.5736	0.2790	0.2905	0.3736	0.3876	0.8919	0.8874
4	0.4345	0.2116	0.2190	0.3147	0.3239	0.9066	0.9029
5	0.3489	0.1693	0.1764	0.3036	0.3158	0.9112	0.9075
6	0.2892	0.1398	0.1467	0.2759	0.2846	0.9180	0.9151
7	0.2452	0.1190	0.1233	0.2668	0.2751	**0.9202**	0.9195

The context benefit in terms of loss (3.0%) is larger than the accuracy improvement (0.07pp). This is expected: loss captures confidence calibration (the model assigns higher probability to correct tokens with context), while accuracy is a harder threshold metric. The model's predictions are more *confident* with context even when the top-1 prediction doesn't change.

9 Implementation Details

9.1 Processor Fix: Fixed-Length Context Padding

A critical implementation issue was discovered and resolved during development. The original processor skipped the context block entirely when `context_text` was an empty string (first word of sentence), producing input sequences of length 128. Samples with context produced sequences of length 148. This length mismatch caused `RuntimeError: stack expects each tensor to be equal size` during DataLoader collation.

The fix ensures that even empty context produces a fixed-length padding block:

Listing 4: Fixed processor: empty context still gets padding block

```
if context_text != "":
    # Tokenize, truncate, pad to max_context_length
    context_padded = context_ids + [sep] + [pad] * pad_needed
    context_attn_mask = [1] * actual_len + [0] * pad_needed
else:
```

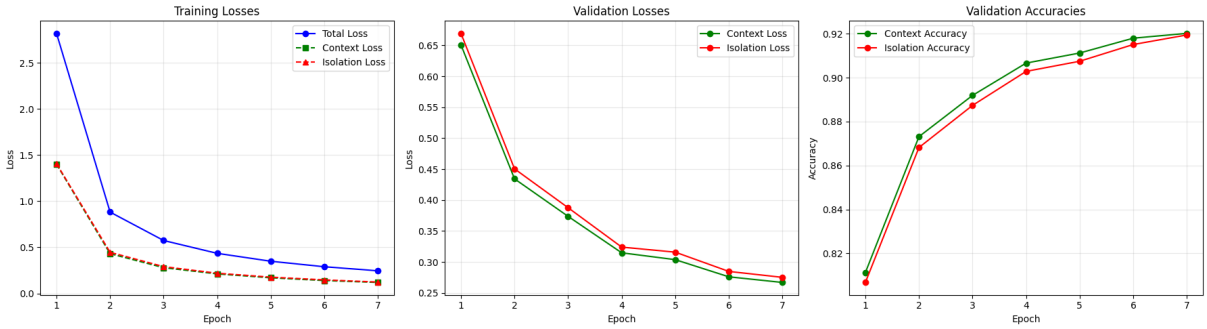

Figure 1: Stage 3 training curves. **Left**: Training losses showing rapid convergence of total, context, and isolation losses. **Center**: Validation losses where context loss (green) remains consistently below isolation loss (red). **Right**: Validation accuracies climbing from 81% to 92%, with context accuracy maintaining a persistent edge.

Table 3: Context benefit measured as $\mathcal{L}_{iso} - \mathcal{L}_{ctx}$ (positive = context helps)

Epoch	Loss Difference	Relative Improvement
1	−0.0186	2.78%
2	−0.0162	3.60%
3	−0.0140	3.61%
4	−0.0092	2.84%
5	−0.0121	3.83%
6	−0.0087	3.06%
7	−0.0082	2.98%

```
    # Empty context: all padding, fully masked out
    context_padded = [pad] * max_context_length  # 20 PAD tokens
    context_attn_mask = [0] * max_context_length  # All masked
```

This guarantees uniform tensor shapes across all samples, regardless of context availability.

9.2 RoPE Sequence Length Adjustment

The original `max_position_embeddings = 256` was insufficient for the RoPE cos/sin tables when context tokens are included. The actual maximum sequence length is:

$$\text{max_seq} = \underbrace{20}_{\text{context}} + \underbrace{128}_{\text{patches}} + \underbrace{128}_{\text{text}} = 276$$

The RotaryEmbedding initialization was adjusted to:

```
max_seq = config.max_position_embeddings + config.max_context_length  # 276
self.rotary_emb = RotaryEmbedding(self.head_dim, max_seq)
```

9.3 Memory Optimization

The dual-path training requires two forward passes per batch, effectively doubling memory consumption. To fit within the L4's 22 GB:

- **Batch size reduced to 16**: Down from 96 (Stage 2) to accommodate the doubled forward passes.
- **bfloat16 mixed precision**: Native L4 support for bfloat16 halves memory for activations without requiring a gradient scaler.

- **Gradient clipping**: Max norm of 1.0 prevents gradient explosion from the combined loss.

10 Discussion

10.1 Effectiveness of Context-Aware Training

The results confirm that inter-word context improves Bengali handwritten text recognition. The 3.0% reduction in validation loss with context is consistent across all epochs, demonstrating that the model has learned to leverage the previous word's text as a disambiguation signal.

The relatively modest accuracy improvement (0.07 percentage points) compared to the loss improvement (3.0%) suggests that context primarily helps with **confidence calibration** rather than changing top-1 predictions. This is still valuable for downstream applications: a more calibrated model produces better-quality beam search outputs and more reliable confidence scores.

10.2 Role of the Margin Penalty

The margin penalty with $\lambda = 0.7$ proved effective. At every epoch, $\mathcal{L}_{ctx} < \mathcal{L}_{iso}$, meaning the model consistently performs better with context. Without the margin penalty, there would be no guarantee that the context path would outperform isolation—the model could potentially learn to ignore or be confused by context tokens.

10.3 RoPE vs. Absolute Positional Embeddings

The transition from absolute positional embeddings to RoPE was seamless, with the model achieving strong performance (92% accuracy) after only 7 epochs of fine-tuning. RoPE's relative position encoding is particularly well-suited for context-aware recognition because:

- The model must attend across variable-length context blocks, patch sequences, and target tokens—relative positions are more meaningful than absolute indices in this heterogeneous sequence.
- RoPE's parameter-free design means no positional embedding weights need to be learned from scratch; only the attention weights are adapted during fine-tuning.

10.4 Limitations and Future Work

- **Single-word context**: Currently only the immediately preceding word is used. Extending to multi-word context (e.g., a sliding window of 2–3 words) could provide richer linguistic signals.
- **Unidirectional context**: Only left-to-right context is used. For offline recognition, bidirectional context (including the following word) could further improve accuracy.
- **Context accuracy gap**: The 0.07pp accuracy improvement is small. Longer training, larger λ, or curriculum learning (gradually increasing context reliance) may widen this gap.
- **Language model integration**: Combining the visual recognizer's context with an external Bengali language model could provide complementary improvements.

11 Conclusion

This report documented the Stage 3 fine-tuning of GraDeT-HTR with Rotary Position Embeddings and context-aware dual-path training. Starting from a Stage 2 pretrained model, we:

1. Replaced absolute positional embeddings with RoPE, converting GPT-2 Conv1D weights to separate Linear projections with proper transposition and splitting.
2. Trained for 7 epochs with a dual-path loss ($\lambda = 0.7$) on an NVIDIA L4 GPU in approximately 3.2 hours.
3. Achieved **92.02%** validation accuracy with context and **91.95%** without, with a consistent 3.0% loss improvement from context across all epochs.

The margin penalty successfully enforced that context always helps, and the RoPE integration maintained strong recognition performance while enabling better relative position encoding for the heterogeneous context–image–text sequence. The resulting model is ready for deployment in the GraDeT-HTR inference pipeline, where sequential word recognition can feed each prediction as context for the next word.

References

[1] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568:127063, 2024. arXiv:2104.09864.